Advanced Machine Learning for Physics student project: RECENTRE

Francesco Passante and Eugenio Nicotina
(Dated: July 10, 2026)

We build a machine learning framework to train, validate, finetune models and perform data analysis for the RECENTRE (REal-time motion CorrEction in magNeTic REsonance) project. We first reproduce the original results of the RECENTRE project based on a GRU network through a config-oriented software design, then we introduce and evaluate more modern machine learning architectures, studying in what regimes they are most effective and whether they're better candidates for real time deployment in MRI machines. Model size, inference latency, robustness to noise and uncertainty calibration are among the many different metrics by which we evaluate the performances of each of the proposed architectures.

I. INTRODUCTION

The RECENTRE project is concerned with real time prediction and correction of head motion parameters. A crucial fact to take into account when building models for this task is the inference latency of the model. The temporal distance between the frame recording and the subsequent next frame prediction must be of the order of 100 ms, so the inference latency must be well below this limit to account for other software- and hardware-related latencies.

The main goals of the present project are to reproduce the original paper's ([]) results, which are based on a GRU network, and evaluate modern machine learning architectures such as transformers, TCNs, and physics-informed models to study whether they are better candidates for the project's task.

Given the large amount of models and the large amount of training runs (different model hyperparameters, different train / test tasks, data augmentation, etc...) we introduced a config-based software design pattern to keep the code and the output data well organized. This way, new models can be implemented and registered in a model registry dictionary; the hyperparameters of the model, the preprocessing pipeline of the data and the training details (epochs, learning rate, ...) can all be specified in a minimal, easy-to-read configuration file. This way, we built a solid end-to-end pipeline from the original raw data and model specification to the full training and evaluation loop.

II. DATASET

The raw dataset is taken from the Human Connectome Project (HCP), which consists in navigator data taken by MRI machines with a sampling frequency of 720 ms. Each 'frame' has 6 motion parameters: 3 of them are translation parameters describing the head position of the patient, the remaining 3 are rotation parameters indicating the inclination of the patient's head. The dataset contains three different scenarios: the "R" (resting) one, where patients were resting while the MRI scan was running, then the "M" (memory) one, where patients were recollecting memories, and the "L" (language) one, where patients were actively talking. There are a total of 3221 time series among the three tasks. Each time series is associated with a patient. The preprocessing pipeline discards 140 time series, so that 1027 patients are all associated with the 3 time series of the "R", "M", "L" tasks. The three tasks have different time lengths. The R data is 1200 frames long, while the M and L are 405 and 316 frames long, respectively. Lots of efforts were put in the thesis and paper ([]) to study different data splits for training and testing, e.g. considering the "cross-task" scenario or "cross-patient" scenario. We build on their findings (no substantial difference in performance among the different scenarios) by:

1. Developing a minimal, simple pipeline to choose training and testing tasks and choosing whether to use different splits of patients

2. Standardizing all results to the same scenario: train on all tasks (R+M+L) and test on all tasks (R+M+L), while keeping disjoint sets of patients to avoid any possible leakage

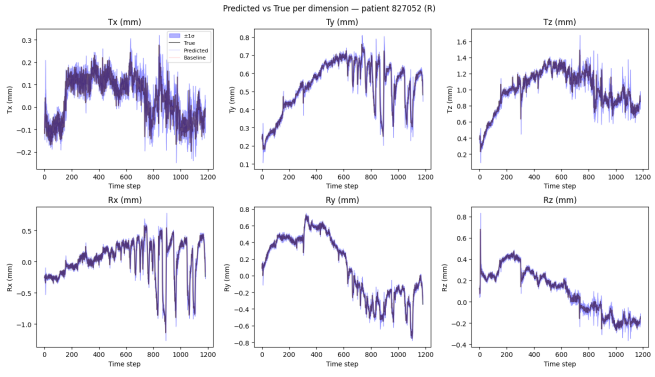

FIG. 1: Example patient time series showing predicted vs. true head-motion parameters across the six degrees of freedom.

A subsequent analysis on physics-informed models has shown that exploiting additional kinematic data such as velocity and acceleration parameters substantially im-

proved test performances. So, 6 additional columns containing velocity parameters and 6 columns containing acceleration parameters for the 6 different dimensions are kept and used.

A. Data augmentation

The dataset contains roughly 2 million input window - frame target pairs, so data augmentation is not a priority for the project. Nevertheless, we investigated whether data augmentation could provide a benefit to model performances. Since the translation and rotation parameters are physically linked, there is no much room for independent transformations of the input data. However, two possible transformations are considered for data augmentation: time reversal and sign flip. These two augmentation techniques can quadruple the initial training dataset.

B. Preprocessing

The preprocessing pipeline consists in extracting the raw data from the HCP files, converting degrees to radians, ensuring a standardized time length for each of the different tasks, and ensuring that each patient is associated with its three runs. The preprocessing pipeline is meant to be run only once, as it builds three processed dictionaries (one for each task) that associate each patient id with its time series for that task. All successive scripts read from the processed datasets.

III. METHODOLOGY

A. Pipeline

All the tested models follow the same protocol:

1. Model definition and registration
2. Training config compilation
3. Training loop
4. Evaluation loop and data analysis
5. Optional fine-tuning

After model definition, typically as a torch module, a training config is necessary to run the training loop. The model hyperparameters, training and testing tasks, split percentages, data augmentation, loss function, training epochs, learning rate and so on are all specified in the training config, so that all the relevant parameters are easy to read, especially for systmeatic comparisons between many different models and runs.

After the training is done, the model weights, the optimizer and scheduler states, training configs, info about the training and testing tasks and patient ids are stored in a checkpoint to be used for testing, finetuning or resuming training.

B. Metrics

The fundamental metric that describes the prediction error of a single frame is called frame displacement (FD). It is defined as

$$FD(t) = \sum_{i=1}^{3} |y_i(t) - \hat{y}_i(t)| + 50\text{mm} \sum_{i=4}^{6} |y_i(t) - \hat{y}_i(t)| \quad (1)$$

it is essentially an absolute error that combines translation error ($i = 1, 2, 3$) and rotation error ($i = 4, 5, 6$), where the radians are scaled by 50mm, the average skull radius. This metric is a single scalar describing the absolute error of the prediction relative to the true position. Since the sampling rate is 720ms, the lag1-model that predicts the last observed position as the next one is already a solid baseline. We'll refer to the lag-1 model as the baseline model. Naturally, we can define the FD_{base} as the frame displacement for the lag1-model:

$$FD_{base}(t) = \sum_{i=1}^{3} |y_i(t) - y_i(t-1)| + 50\text{mm} \sum_{i=4}^{6} |y_i(t) - y_i(t-1)| \quad (2)$$

Another useful metric is the FD_{gain}, that is the relative difference between FD_{base} and FD_{model}:

$$FD_{gain} = \frac{FD_{base} - FD_{model}}{FD_{base}} \quad (3)$$

We aim for large, positive FD_{gain}.

Another useful metric is the percentage of frames that is improved (i.e. lower frame displacement) with respect to the baseline. Analogously, we can look at the percentage of patients with an average $FD_{gain} > 0$.

C. Loss function

We use the same form of the loss function as the original work:

$$L = L_{NLL} - \beta FD_{gain} \quad (4)$$

Where L_{NLL} is the gaussian negative log likelihood term that is needed for both prediction accuracy and uncertainty calibration.

Using FD_{gain} in the loss is a way to penalize the model if it does worse than the baseline model, especially when the baseline model is performing well in that particular frame. This loss term is essentially specifying that we'd rather accept large FDs in frames where the baseline has poor performance than have frames where the model is worse than the baseline.

We also reduce the learning rate on loss plateau and use early stopping.

D. Finetuning

Finetuning in this project is intended as a per-task, per-patient finetuning. After a model has been pretrained on many different time series, it can be additionally fine-tuned on the first part of a time series so that the model can be adapted to the particularities of the patient and task under observation. The original work has found a substantial improvement in test performance after finetuning, relative to the pre-trained generalist model.

The finetuning implementation idea is the same as the original work one. It uses a modified loss function:

$$L = L_{MSE} - \beta FD_{gain} + \lambda L_{L2SP} \tag{5}$$

It differs from the pre-training loss function because it lacks an uncertainty calibration term (which is fine because the uncertainty head is frozen) and it has a L^2 starting point (L2SP) term that penalizes significant drifts from the pre-trained model:

$$L_{L2SP} = \sum_i (w_i - w_i^0)^2 \tag{6}$$

where w_i^0 is the i-th pretrained model weight.

Our own implementation aims to standardize this procedure across all the implemented models, and make all the finetuning parameters readily accessible in a dedicated finetuning config file. Each model inherits from a TunableModel base class that automatically splits the model parameters into a frozen variance head, a higher learning rate mean head and a lower learning rate body. The learning rates and the dropout rate are configurable in the finetuning config.

IV. REPRODUCTION OF PREVIOUS RESULTS

The first part of the project is concerned with reproducing the results of the previous thesis and paper. First, we refactored the codebase, fixed minor bugs and organized it in a config-oriented design, with standardization of train/test scenarios, model definitions, finetuning pipeline, model / data / train hyperparameters, data analysis and visualizations.

Then, we focused on reproducing these GRU results from the previous work:

1. Paper main result: $R^2 > 0.87$ for all the 3 tasks and 6 dimensions.
2. Thesis result: roughly 70% frames have $FD_{gain} > 0$.
3. Thesis result: positive FD_{gain} frames percentage rises to 75% for the finetuned model.

We first reproduced the original results by considering the per-task training and testing, but we soon decided to directly improve these results by adopting some fairly obvious refinements on top of the original structure. First, we dropped the cross-task scenario debate and decided to train and test a single model on all three tasks. We obviously used different patients for the train, validation and test splits to avoid any data leakage. The hyperparameters of the model are left the same. The results are visible in Fig. (**??**)

We also reproduced the per-patient fine-tuning result of the previous work. Fig. (**??**) summarizes the effect of fine-tuning, and Fig. (**??**) compares each patient before and after fine-tuning.

V. RESULTS

A. β scan

The previous work used a fixed (arbitrary) choice of $\beta = 0.1$. We investigated the performances of the original GRU model as β is swept over many orders of magnitude. In particular, we focused on the most relevant metrics that could sharply depend on the relative importance between the negative log likelihood and the FD_{gain} term. Since the negative log likelihood term is the only one explicitly constraining the uncertainty of the prediction,we expect that as β grows, the uncertainty calibration will get worse. On the other hand, FD_{gain} and the percentage of improved frames (relative to baseline) should rise with β. Our empirical results are summarized in Fig. (**??**).

As expected, uncertainty calibration degrades as β grows near and past $\beta = 1$, while FD_{gain} rises and then decreases again when FD_{gain} is over-weighted in the loss. This is probably an overfitting phenomenon specific to the FD_{gain} term: even though the loss function at very high β is essentially a maximization of FD_{gain}, this doesn't generalize well to the validation/test set. Across different scans, the three values $\beta = 0.1, 0.5, 1$ showed the best performances. We chose to use $\beta = 0.5$ as a middle ground between the optimally calibrated but lower FD_{gain} focused $\beta = 0.1$ and the less calibrated but higher FD_{gain} focused $\beta = 1$.

TABLE I: Effect of the jerk-penalty weight γ on test-set performance (GRU, sequence length 32).

γ	Mean FD	Mean FD_{gain}	% Frames > base
0	0.1268	0.1778	78.4
0.001	0.1267	0.1785	78.5
0.01	0.1267	0.1767	78.2
0.1	0.1280	0.1786	79.0
0.3	0.1273	0.1798	78.8
0.5	0.1285	0.1769	78.7
0.7	0.1312	0.1693	78.5
1	0.1363	0.1514	77.3

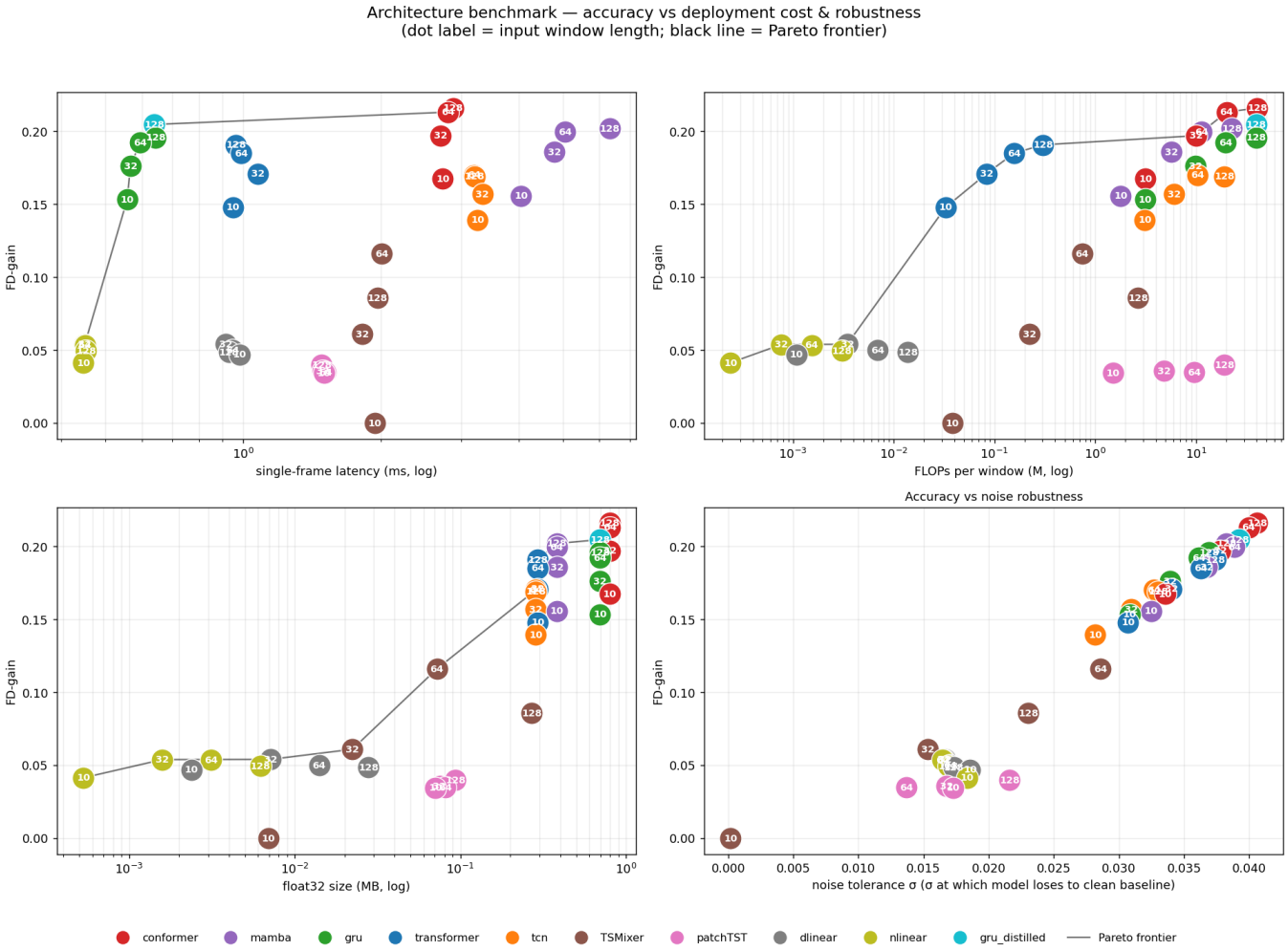

FIG. 10: Pareto-optimal frontier visualizations including the conformer-distilled GRU. The distilled GRU attains a substantially higher FD_{gain} than the vanilla GRU while retaining the GRU's very low latency, moving it toward the accuracy of the heavier conformer/mamba experts.